DOCUMENTACIÓN TÉCNICA

Medallion Data Platform

Diseño Arquitectónico de la Solución

Plataforma Empresarial de Datos Multi-Cloud

Patrón Medallion · Apache Spark · Kubernetes · Multi-Cloud

Versión 7.0

Abril 2026

Índice

Índice 2
1. Resumen Ejecutivo 5
1.1 Objetivos del Diseño 5
1.2 Capacidades Clave 5
2. Contexto y Dominio de Negocio 7
2.1 Dominios Atendidos 7
2.2 Datasets Principales 7
3. Visión General de la Arquitectura 8
3.1 Capas Lógicas 8
3.2 Stack Tecnológico 9
3.3 Modos de Ejecución 9
4. Patrón Arquitectónico Medallion 11
4.1 Capas del Patrón 11
4.2 Características por Capa 11
4.2.1 Capa Bronze 11
4.2.2 Capa Silver 12
4.2.3 Capa Gold 12
4.3 Flujo de Datos End-to-End 13
5. Arquitectura del Motor ETL 14
5.1 Estructura Modular 14
5.2 Motor de Ejecución DAG 15
5.2.1 Topología del DAG 15
5.2.2 Tipado de Errores 16
5.2.3 Tareas Críticas vs No Críticas 16
5.3 Patrón Circuit Breaker 16
5.4 Catálogo Único: TableRegistry 17
5.5 Workflows Especializados 17
6. Infraestructura y Deployment 18
6.1 IBM Cloud 18
6.1.1 Recursos Principales 18
6.1.2 Configuraciones por Ambiente 18
6.2 Spark sobre Kubernetes 19
6.2.1 Manifests Kubernetes 19

6.2.2 Características Operacionales 20
6.3 Lakehouse on-premise (Hadoop) 20
6.4 Azure 21
7. Orquestación de Datos 22
7.1 Azure Data Factory 22
7.2 Linked Services 22
7.3 Distribución de Actividades 23
8. Capa Analítica y de Consumo 24
8.1 Modelo Dimensional Star Schema 24
8.2 Medidas DAX 25
8.3 Dashboards 25
8.4 Visualizaciones Embebidas 25
9. Estrategia de Seguridad 27
9.1 Modelo de Capas 27
9.2 Security Context de Contenedores 28
9.3 NetworkPolicy Zero-Trust 28
9.4 Encriptación en Tránsito 28
9.5 Seguridad en CI/CD 29
9.6 IAM y Mínimo Privilegio 29
10. Continuous Integration y Continuous Delivery 30
10.1 Etapas del Pipeline 30
10.2 Recursos Asignados 31
10.3 Triggers 31
10.4 Workspaces y Parámetros 31
11. Observabilidad y Monitoreo 32
11.1 Componentes de Observabilidad 32
11.2 Alertas Configuradas 32
12. Consideraciones Transversales 33
12.1 Idempotencia y Checkpoints 33
12.2 Feature Flags 33
12.3 ViewScope (Loan Pattern) 33
12.4 Estrategia de Partitioning 33
12.5 Disaster Recovery 34
13. Estructura del Repositorio 35
13.1 Directorios Principales 35

13.2 Documentación Modular 35
14. Conclusiones del Diseño 37
14.1 Fortalezas Arquitectónicas 37
14.2 Áreas Potenciales de Mejora 37
14.3 Madurez de la Solución 37

1. Resumen Ejecutivo

Este documento describe el diseño arquitectónico técnico de la Medallion Data Platform, una plataforma empresarial de datos diseñada para soportar análisis de negocio en dos dominios verticales: comercio electrónico minorista (retail de bicicletas y componentes) y operaciones de extracción minera. La solución implementa el patrón arquitectónico Medallion (Raw → Bronze → Silver → Gold) sobre una infraestructura híbrida y multi-cloud que combina recursos en Azure, IBM Cloud y entornos on-premise basados en Hadoop.

La plataforma está construida con un enfoque cloud-native, contenerizada en Docker, orquestada con Kubernetes, e instrumentada con observabilidad de extremo a extremo mediante Prometheus y Sysdig. El motor de procesamiento principal es Apache Spark 3.3.1 escrito en Scala, complementado con notebooks PySpark para Databricks y Azure Data Factory para la orquestación cross-cloud.

1.1 Objetivos del Diseño

- Implementar un Lakehouse moderno que combine la flexibilidad de un Data Lake con las garantías ACID de un Data Warehouse.
- Garantizar portabilidad de cargas de trabajo entre proveedores cloud y entornos on-premise mediante contenerización y abstracción de almacenamiento.
- Aplicar principios DevSecOps con CI/CD automatizado, Infrastructure as Code, scanning de seguridad y políticas zero-trust.
- Habilitar análisis de negocio mediante un modelo dimensional en estrella con 57 medidas DAX distribuidas en 6 dashboards de Power BI.
- Proveer un motor de ejecución resiliente con tolerancia a fallos, reintentos exponenciales, circuit breakers y checkpoints idempotentes.

1.2 Capacidades Clave

Capacidad	Implementación
Procesamiento batch a escala	Apache Spark 3.3.1 con dynamic allocation
Almacenamiento transaccional	Delta Lake con garantías ACID
Catálogo de metadatos	Apache Hive Metastore (Thrift 9083)
Orquestación de tareas	DAG Engine propio + Azure Data Factory
Ejecución programada	Kubernetes CronJob + IBM Analytics Engine
CI/CD	Tekton Pipelines de 9 etapas

Capacidad	Implementación
Infrastructure as Code	Terraform (IBM) + PowerShell (Azure)
Inteligencia de negocio	Power BI con star schema y 57 medidas DAX
Visualización embebida	JFreeChart (10 charts PNG generados)
Monitoreo	Prometheus + Sysdig + IBM Activity Tracker

2. Contexto y Dominio de Negocio

La plataforma soporta dos unidades de negocio independientes pero compartiendo la misma infraestructura técnica, lo que permite economías de escala en operación y desarrollo.

2.1 Dominios Atendidos

Dominio	Descripción	Fuentes Primarias
Retail	E-commerce de bicicletas y componentes con análisis de ventas, rentabilidad y segmentación de clientes RFM.	SQL Server (dbRetail), CSV batch
Minería	Operaciones de extracción mineral con seguimiento de productividad por operador, eficiencia de equipos y producción geográfica.	CSV batch, sensores operacionales

2.2 Datasets Principales

La capa Raw alberga 7 datasets fuente que constituyen la base de toda la plataforma:

Dataset	Dominio	Descripción
Categoria.csv	Retail	Categorías maestras de productos
Subcategoria.csv	Retail	Subcategorías agrupadas por categoría
Producto.csv	Retail	Catálogo maestro de productos
VentasInternet.csv	Retail	Transacciones de venta online
Sucursales.csv	Retail	Ubicaciones físicas de sucursales
FactMine.csv	Minería	Hechos de producción minera
Mine.csv	Minería	Operaciones detalladas por truck/proyecto

3. Visión General de la Arquitectura

La arquitectura de la solución se organiza en seis capas lógicas claramente separadas, cada una con responsabilidades específicas y contratos de interfaz bien definidos. Esta separación permite evolucionar componentes individualmente sin afectar al resto del sistema.

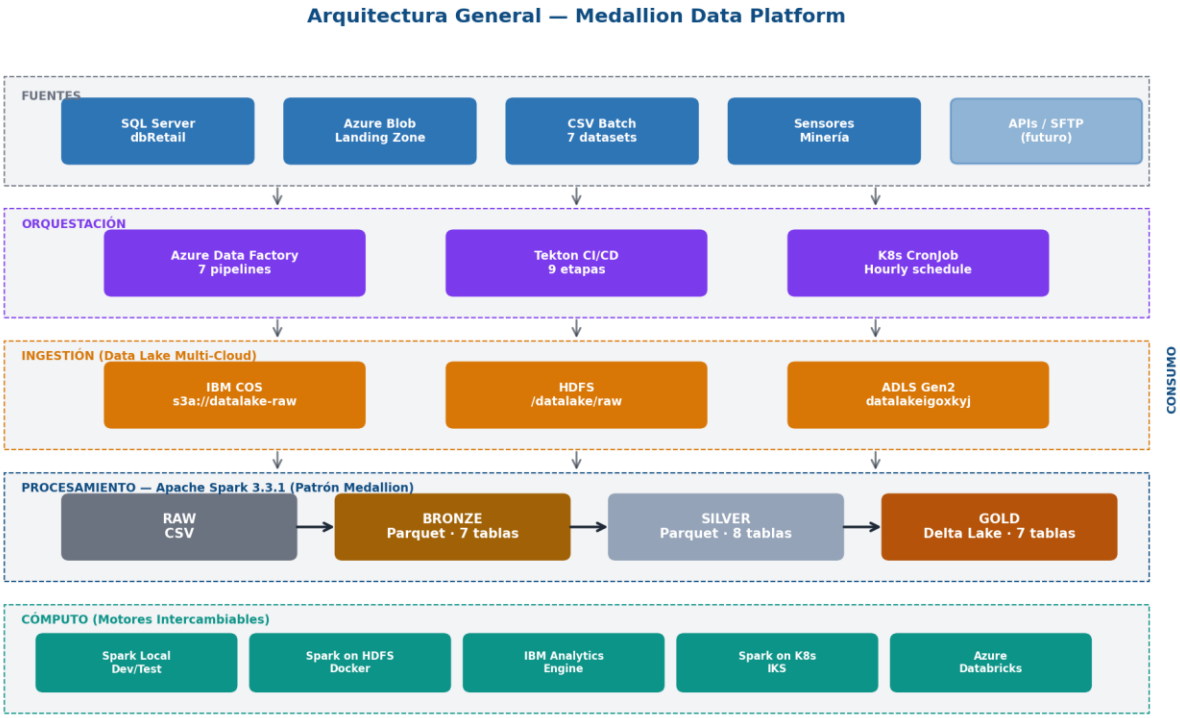

Figura 1. Arquitectura general de la plataforma — vista de capas con flujo de datos y motores de cómputo intercambiables.

3.1 Capas Lógicas

#	Capa	Responsabilidad	Tecnologías
1	Fuentes	Sistemas origen de datos transaccionales y archivos batch	SQL Server, CSV, Azure Blob
2	Orquestación	Coordinación, scheduling y gobernanza de pipelines	ADF, Tekton, K8s CronJob, DAG Engine
3	Ingestión	Almacenamiento de datos crudos en buckets multi-cloud	COS, ADLS Gen2, HDFS
4	Procesamiento	Transformación distribuida siguiendo el patrón Medallion	Apache Spark 3.3.1 (Scala / PySpark)
5	Cómputo	Motores de ejecución intercambiables	Spark Local, IBM AE, IKS, Databricks

#	Capa	Responsabilidad	Tecnologías
6	Consumo	Capa de presentación analítica y catálogo	Power BI, Hive Catalog, JFreeChart

3.2 Stack Tecnológico

Componente	Tecnología	Versión / Notas
Lenguaje principal	Scala	2.12 sobre JVM 11
Lenguaje secundario	Python (PySpark)	Notebooks Databricks
Motor de procesamiento	Apache Spark	3.3.1
Formato analítico	Delta Lake	ACID transactions
Serialización	Kryo	Buffer 256 MB
Runtime de contenedores	Docker	Multi-stage build
Orquestación K8s	Kubernetes (IKS)	CronJob batch/v1
CI/CD	Tekton Pipelines	9 etapas
IaC en IBM Cloud	Terraform	Provider ibm 1.71
Orquestación de datos	Azure Data Factory	7 pipelines
Inteligencia de negocio	Power BI	57 medidas DAX
Monitoreo	Prometheus + Sysdig	ServiceMonitor
Almacenamiento distribuido	Apache Hadoop HDFS	3.3.4
Catálogo de metadatos	Apache Hive Metastore	Thrift 9083
Bases de datos relacionales	SQL Server / IBM Db2	Cloud managed
Visualización embebida	JFreeChart	10 charts PNG generados

3.3 Modos de Ejecución

El pipeline puede ejecutarse en tres modos operativos distintos, auto-detectados en tiempo de arranque por el componente ContextBuilder mediante variables de entorno. Esta capacidad permite usar el mismo binario desde el laptop de un desarrollador hasta producción cloud.

Modo	Trigger (env var)	Caso de Uso
IBM Analytics Engine Serverless	AE_INSTANCE_ID presente	Producción cloud sin gestión de clusters

Modo	Trigger (env var)	Caso de Uso
HDFS + Hive Lakehouse	HDFS_URI presente	On-premise / staging con Docker Compose
Local filesystem	Default (sin env vars)	Desarrollo y pruebas en laptop

4. Patrón Arquitectónico Medallion

El patrón Medallion es la columna vertebral conceptual de la solución. Organiza los datos en cuatro capas progresivamente refinadas, donde cada capa aporta valor incremental sobre la anterior. Esta estratificación habilita reproducibilidad, debuggeabilidad y evolución controlada de los modelos analíticos.

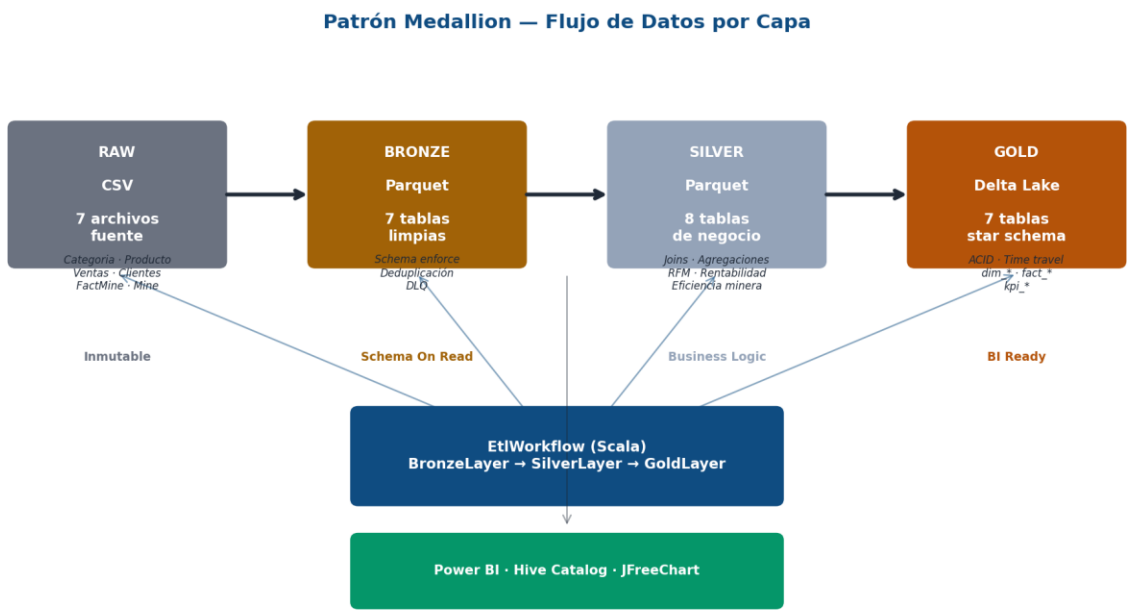

Figura 2. Patrón Medallion — flujo de datos progresivo entre las cuatro capas con sus formatos de almacenamiento.

4.1 Capas del Patrón

Capa	Formato	Propósito	Tablas
RAW	CSV	Snapshot inmutable de los datos fuente tal como llegaron	7 archivos
BRONZE	Parquet	Limpieza estructural, schema enforcement, deduplicación, columnas de auditoría	7 tablas
SILVER	Parquet	Lógica de negocio, agregaciones intermedias, enriquecimiento	8 tablas
GOLD	Delta Lake	Modelo dimensional star schema listo para BI con garantías ACID	7 tablas

4.2 Características por Capa

4.2.1 Capa Bronze

La capa Bronze realiza la primera transformación de los datos crudos. Sus responsabilidades técnicas son:

- Aplicación de schema enforcement mediante StructType definidos en CsvSchemas.
- Deduplicación por claves naturales declaradas en TableRegistry (por ejemplo, Cod_Producto para productos).
- Filtrado de filas con nulos en campos críticos.
- Adición de columnas de auditoría como timestamp de ingesta.
- Implementación de Dead Letter Queue para registros que fallan la validación.
- Conversión del formato CSV a Parquet, ganando compresión columnar y predicate pushdown.

4.2.2 Capa Silver

La capa Silver materializa lógica de negocio reutilizable. Las 8 tablas Silver atienden ambos dominios:

Tabla Silver	Dominio	Lógica Aplicada
catalogo_productos	Retail	Join jerárquico producto → subcategoría → categoría
ventas_enriquecidas	Retail	Ventas con jerarquía de producto, márgenes y tipo de envío
resumen_ventas_mensuales	Retail	Agregación temporal por mes y categoría
rentabilidad_producto	Retail	Revenue, margen y ranking por producto
segmentacion_clientes	Retail	Segmentación RFM (Recency, Frequency, Monetary)
produccion_operador	Minería	Producción agregada por operador minero
eficiencia_minera	Minería	Eficiencia operacional por truck y proyecto
produccion_por_pais	Minería	Producción agregada geográficamente

4.2.3 Capa Gold

La capa Gold es el destino final del pipeline ETL y la fuente exclusiva del consumo analítico. Está implementada en Delta Lake, lo que aporta transacciones ACID, time travel y optimizaciones automáticas como auto-optimize y optimized writes. Su modelo es un star schema clásico con tablas de hechos y dimensiones.

Tabla Gold	Tipo	Descripción
dim_producto	Dimensión	Producto con clasificación de rentabilidad y rotación
dim_cliente	Dimensión	Cliente con segmentación RFM y LTV calculado
dim_operador	Dimensión	Operador minero con ranking de productividad
fact_ventas	Hecho	Transacciones de venta con métricas financieras

Tabla Gold	Tipo	Descripción
fact_produccion_minera	Hecho	Hechos de producción con métricas de eficiencia
kpi_ventas_mensuales	Hecho agregado	KPIs ejecutivos pre-calculados (MoM, YTD)
kpi_mineria	Hecho agregado	KPIs operacionales de minería

4.3 Flujo de Datos End-to-End

El recorrido completo de un dato desde la fuente hasta el dashboard atraviesa los siguientes hitos secuenciales:

1. Ingestión: datos extraídos de SQL Server o cargados desde CSV se depositan en la zona Raw del data lake (COS, ADLS o HDFS según el modo de ejecución).

2. Bronze: el componente BronzeLayer lee los CSV con schema explícito, deduplica y persiste en Parquet particionado.

3. Silver: SilverLayer ejecuta joins, agregaciones y enriquecimientos aplicando reglas de negocio.

4. Gold: GoldLayer materializa las tablas dimensionales y de hechos en Delta Lake con garantías ACID.

5. Catálogo: las tablas Gold se registran en Hive Metastore para consulta SQL externa.

6. Consumo: Power BI conecta a Gold mediante DirectQuery o Import, aplica las 57 medidas DAX y renderiza los 6 dashboards.

5. Arquitectura del Motor ETL

El componente más sofisticado de la plataforma es el motor ETL implementado en Scala. Su diseño aplica patrones de ingeniería de software avanzados para garantizar resiliencia, observabilidad y mantenibilidad. La versión actual del orquestador (v7.0) reduce el método main() a apenas 15 líneas, delegando toda la complejidad a componentes especializados con responsabilidades únicas.

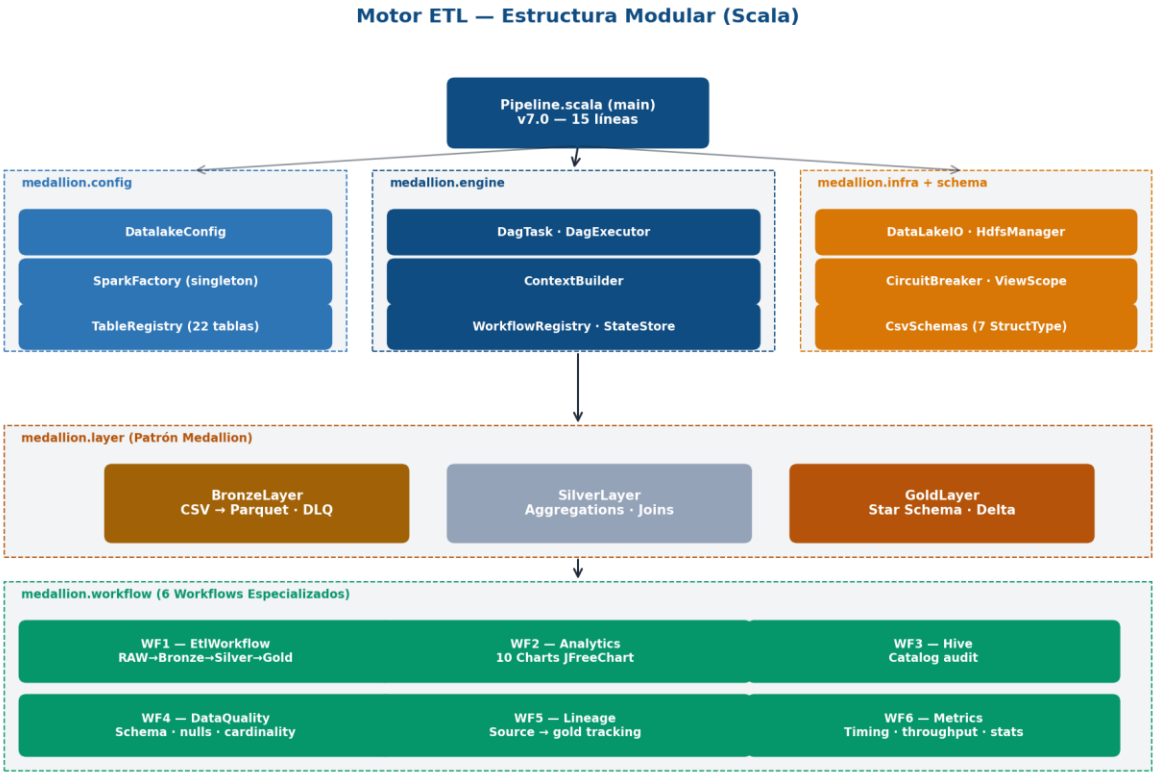

Figura 3. Estructura modular del motor ETL — paquetes Scala y sus responsabilidades específicas.

5.1 Estructura Modular

El código está organizado en seis paquetes Scala que reflejan los conceptos del dominio de un pipeline de datos:

Paquete	Componentes	Responsabilidad
medallion (root)	Pipeline.scala	Punto de entrada, orquestación de alto nivel
medallion.config	DatalakeConfig, SparkFactory, TableRegistry, IbmCloudConfig	Configuración centralizada y singletons
medallion.schema	CsvSchemas	Definiciones de StructType para los 7 datasets

Paquete	Componentes	Responsabilidad
medallion.engine	DagTask, DagExecutor, ContextBuilder, WorkflowRegistry, PipelineStateStore	Motor de ejecución declarativa con DAG
medallion.infra	DataLakeIO, HdfsManager, CircuitBreaker, ViewScope	Abstracciones de I/O y resiliencia
medallion.layer	BronzeLayer, SilverLayer, GoldLayer	Implementación del patrón Medallion
medallion.workflow	EtlWorkflow, AnalyticsWorkflow, HiveWorkflow, DataQualityWorkflow, LineageWorkflow, MetricsWorkflow	Casos de uso especializados

5.2 Motor de Ejecución DAG

El corazón del orquestador es DagExecutor, un motor de ejecución declarativa de tareas con dependencias. Su estrategia de ejecución sigue cinco pasos:

- **Validación:** verifica que el grafo dirigido no contenga ciclos.
- **Identificación:** calcula el conjunto de tareas listas (sin dependencias pendientes).
- **Ejecución paralela:** lanza tareas concurrentemente con un thread pool controlado por el parámetro parallelism.
- **Re-evaluación:** al completarse cada tarea, re-evalúa qué nuevas tareas quedan desbloqueadas.
- **Finalización:** espera con timeout configurable (default 15 minutos) hasta completar todas las tareas.

5.2.1 Topología del DAG

La topología del pipeline ETL principal se representa como:

```
COS_UPLOAD → ETL → [QUALITY || LINEAGE || ANALYTICS || DB2_EXPORT] → HIVE_AUDIT →
METRICS
```

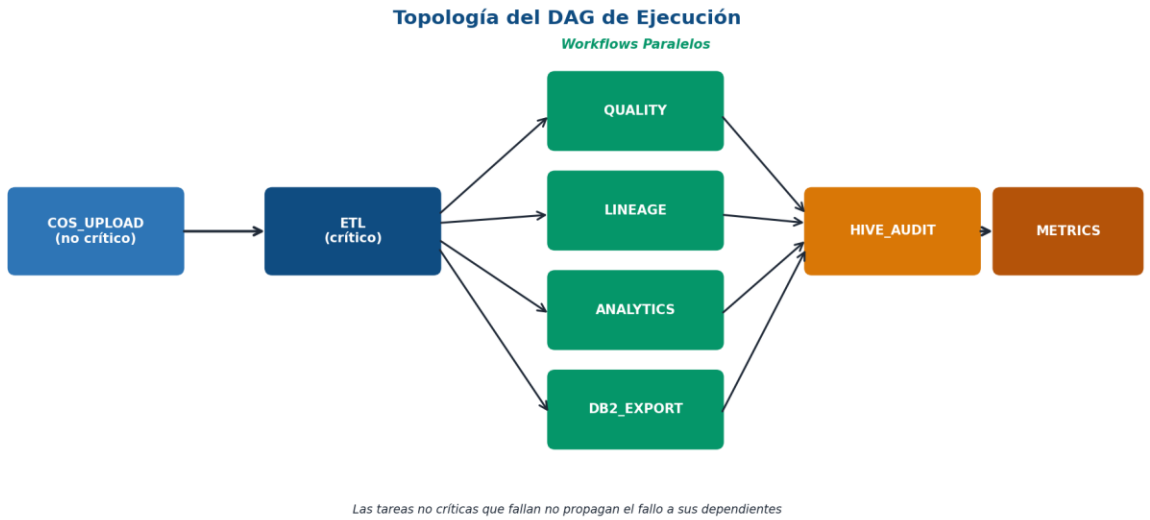

Figura 4. Topología del DAG de ejecución — workflows paralelos tras el ETL principal.

Los workflows entre corchetes ejecutan en paralelo tras la finalización del ETL principal, maximizando el throughput. La operación final METRICS reporta tiempos, throughput y tabla de bottleneck.

5.2.2 Tipado de Errores

El motor implementa errores tipados mediante un sealed trait TaskError con tres variantes que dictan la política de retry:

Tipo de Error	Comportamiento	Caso de Uso
Transient	Reintenta con backoff exponencial (2s × intento)	Errores transitorios de red, throttling
Fatal	Falla inmediatamente sin reintento	Errores de configuración, datos corruptos
Skippable	Marca como CompletedWithWarning y continúa	Tablas opcionales, validaciones blandas

5.2.3 Tareas Críticas vs No Críticas

Cada DagTask tiene un atributo critical: las tareas no críticas que fallan no propagan el fallo a sus dependientes, lo que permite que workflows de soporte (como métricas o data quality en ambientes de desarrollo) no bloqueen el pipeline principal.

5.3 Patrón Circuit Breaker

Para protegerse contra fallos sistémicos en operaciones de I/O (HDFS, COS, Db2), el motor implementa el patrón Circuit Breaker con tres estados:

Estado	Comportamiento	Transición
CLOSED	Operaciones normales, cuenta fallos consecutivos	→ OPEN cuando failures ≥ threshold (default 3)
OPEN	Rechaza operaciones inmediatamente sin intentarlas	→ HALF_OPEN tras resetTimeMs (default 60 s)
HALF_OPEN	Permite una operación de prueba para verificar recuperación	→ CLOSED si éxito · → OPEN si falla

5.4 Catálogo Único: TableRegistry

TableRegistry actúa como fuente única de verdad para todas las tablas del pipeline. Centraliza nombres, formatos, dependencias para linaje, claves de deduplicación y schemas CSV. Esta centralización elimina la duplicación que existía previamente en Pipeline, EtlWorkflow, HiveWorkflow, LineageWorkflow, SilverLayer y GoldLayer.

Cada tabla se modela con el case class:

```
case class TableDef(   name: String,   layer: Layer,                  // Raw |
Bronze | Silver | Gold   format: Format,                // Csv | Parquet | Delta
sources: Seq[String],          // Para linaje y dependencias   deduplicateKeys:
Seq[String],  // Claves naturales (Bronze)   csvFileName: Option[String],   //
Para ingesta inicial   csvSchema: Option[StructType]  // Schema enforcement )
```

5.5 Workflows Especializados

Sobre el motor DAG se construyen seis workflows especializados que implementan los casos de uso operativos:

Workflow	ID	Función Principal
EtlWorkflow	ETL	Pipeline principal RAW → Bronze → Silver → Gold (crítico)
AnalyticsWorkflow	ANALYTICS	Generación de 10 charts JFreeChart en PNG
HiveWorkflow	HIVE_AUDIT	Auditoría y registro en Hive Catalog
DataQualityWorkflow	QUALITY	Validación de schema, nulos y cardinalidad
LineageWorkflow	LINEAGE	Tracking source → gold para gobernanza
MetricsWorkflow	METRICS	Timing, conteos, throughput y bottleneck

6. Infraestructura y Deployment

La plataforma adopta un enfoque multi-cloud que permite ejecutar las mismas cargas de trabajo en distintos proveedores y entornos. La infraestructura se gestiona íntegramente como código mediante Terraform (IBM Cloud) y PowerShell (Azure), garantizando reproducibilidad y trazabilidad.

6.1 IBM Cloud

El despliegue principal en IBM Cloud comprende 36 recursos provisionados con Terraform, organizados en una arquitectura que abarca networking, almacenamiento, cómputo, seguridad, observabilidad e integración continua.

6.1.1 Recursos Principales

Categoría	Recurso	Función
Networking	VPC + Subnet + Public Gateway + Security Group + ACL	Aislamiento de red y conectividad controlada
Cómputo	IKS Cluster (Kubernetes)	Hosting del CronJob de Spark
Cómputo	IBM Analytics Engine Serverless	Ejecución de jobs Spark sin gestión de cluster
Almacenamiento	Cloud Object Storage (4 buckets: raw / bronze / silver / gold)	Data lake principal
Almacenamiento	COS Logs Bucket (opcional)	Centralización de logs
Datos	IBM Db2 on Cloud	Base de datos relacional para export final
ETL	DataStage	ETL gráfico complementario
Seguridad	Key Protect (opcional)	Gestión de claves AES-256
Seguridad	Secrets Manager (opcional)	Vault de credenciales
Identidad	IAM Access Groups (data-engineers, data-analysts)	RBAC con políticas mínimo privilegio
Observabilidad	Activity Tracker, Log Analysis, Sysdig (opcionales)	Auditoría, logs y métricas
CI/CD	Continuous Delivery + Toolchain + GitHub Tool	Pipeline Tekton conectado al repo

6.1.2 Configuraciones por Ambiente

Terraform soporta tres ambientes con archivos .tfvars dedicados:

Ambiente	Workers	Plan	Observabilidad	Seguridad
dev	1	Free / Lite	Deshabilitada	CIDR abierto
staging	2	Standard	Parcial	Standard
prod	3	Standard	Completa (Sysdig)	Key Protect activo

6.2 Spark sobre Kubernetes

El despliegue Spark on Kubernetes implementa un CronJob production-grade en el namespace dedicado spark-medallion. La arquitectura incluye init containers para preflight checks, dynamic allocation de executors, network policies zero-trust y monitoreo Prometheus.

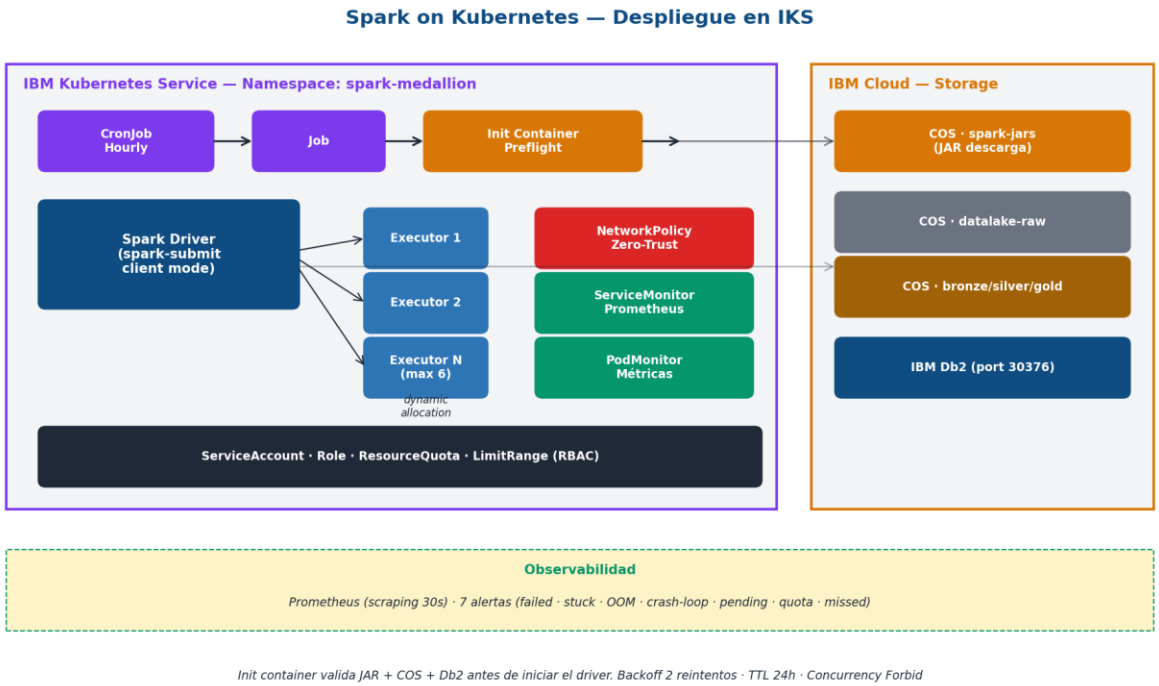

Figura 5. Despliegue Spark on Kubernetes — CronJob, driver, executors dinámicos y conexiones a IBM Cloud.

6.2.1 Manifests Kubernetes

Archivo	Recurso K8s	Función
namespace.yaml	Namespace	spark-medallion con labels de entorno
rbac.yaml	ServiceAccount, Role, ResourceQuota, LimitRange	RBAC y cuotas de recursos
secrets.yaml	Secret (template envsubst)	COS, Db2, IBM Cloud API credentials

Archivo	Recurso K8s	Función
configmaps.yaml	ConfigMap	Spark config, scripts de preflight/healthcheck
cronjob.yaml	CronJob batch/v1	Init container, driver, dynamic allocation, probes
network-policy.yaml	NetworkPolicy	Zero-trust ingress / egress
service.yaml	Service ClusterIP	Spark UI (4040), driver RPC (7078), block manager (7079)
monitoring.yaml	ServiceMonitor, PodMonitor, PrometheusRule	Scraping de métricas y 7 alertas

6.2.2 Características Operacionales

Resiliencia

- Init container preflight: verifica disponibilidad del JAR y conectividad a COS y Db2 antes de iniciar el driver.
- Dynamic allocation con rango de 1 a 6 executors según la carga.
- Concurrency policy Forbid: evita superposición de jobs.
- Backoff limit de 2 reintentos antes de fallar definitivamente.
- TTL cleanup: jobs completados se eliminan automáticamente tras 24 horas.
- History limits: conserva los últimos 3 jobs exitosos y 3 fallidos.

Observabilidad

- ServiceMonitor de Prometheus: scraping del driver cada 30 segundos.
- PodMonitor para executors efímeros.
- PrometheusRule con 7 alertas: job failed, stuck, missed, OOM risk, crash-loop, pending, quota.
- Liveness y readiness probes configurados en el container del driver.

Performance

- Spark defaults optimizados: serializador Kryo, AQE habilitado, broadcast threshold de 256 MB.
- S3A tuning: fast upload, output committer v2, connection pool de 200.
- Delta Lake con auto-optimize y optimized writes habilitados.
- Resource requests y limits granulares por componente.

6.3 Lakehouse on-premise (Hadoop)

Para ejecuciones locales o en datacenter, la plataforma soporta un stack Hadoop completo orquestado con Docker Compose. Este modo replica la funcionalidad cloud usando HDFS como almacenamiento, Hive Metastore como catálogo y MySQL como backend de Hive.

Componente	Función	Puerto Expuesto
NameNode	Maestro de HDFS, gestión de bloques	9870 (UI), 9000 (RPC)
DataNode	Almacenamiento físico de bloques HDFS	9864 (UI), 9866 (transfer)
Hive Metastore	Catálogo de metadatos vía Thrift	9083
MySQL	Backend de Hive Metastore	3306
YARN ResourceManager	Gestión de recursos para MapReduce	8088
Job History Server	Historia de jobs MapReduce	10020

La estructura de directorios en HDFS replica la organización del data lake cloud, con las rutas /datalake/raw, /datalake/bronze, /datalake/silver y /datalake/gold conteniendo las correspondientes capas Medallion, más /datalake/checkpoints para los archivos de estado del DAG executor.

6.4 Azure

El stack Azure aporta capacidades complementarias de orquestación y procesamiento interactivo. Provisionado mediante un script PowerShell y publicado por Azure Pipelines, incluye los siguientes componentes en el resource group Rgfedericopfund:

Componente	Recurso Azure	Función
Fuente OLTP	Azure SQL Server (sqlservergrupo1 / dbRetail)	Base de datos transaccional retail
Landing Zone	Azure Blob Storage (storageblobsigoxkyj)	Zona de aterrizaje de CSV
Data Lake	Azure Data Lake Storage Gen2 (datalakeigoxkyj)	Almacenamiento jerárquico (HNS)
Orquestación	Azure Data Factory (7 pipelines)	Orquestación cross-cloud
Procesamiento	Azure Databricks (cluster 1123-103841)	Notebooks PySpark interactivos
CI/CD	Azure Pipelines (windows-latest)	Provisionamiento PowerShell

7. Orquestación de Datos

La capa de orquestación coordina el movimiento de datos entre sistemas y la ejecución de transformaciones. Coexisten dos orquestadores complementarios: Azure Data Factory para flujos cross-system y el DAG Engine propio para la ejecución interna del ETL Spark.

7.1 Azure Data Factory

ADF gestiona 7 pipelines que cubren la extracción desde SQL Server, el aterrizaje en Blob Storage, la replicación a Data Lake Gen2 y la invocación de notebooks Databricks.

Pipeline	Propósito
Pipeline_llenado_db	Inicialización de la base de datos: validación de schema y CREATE TABLE condicional mediante Lookup + IfCondition + Script
Pipeline_extraccion	Extracción SQL Server → CSV en Blob Storage iterando con ForEach + Copy Activity
Copy_data_sql	Copia batch de múltiples tablas SQL Server a Azure Storage con parametrización dinámica
Busqueda de tablas CSV	Descubrimiento dinámico de archivos CSV en el landing zone vía GetMetadata
Copy data SQL to CSV1	Replicación con SQL Partition para lectura paralela de tablas grandes
ETL	Pipeline principal: extracción de metadata, gestión de variables y ejecución de notebook Databricks
Pipeline auxiliar	Tareas de soporte y reprocesamiento manual

7.2 Linked Services

Los pipelines de ADF se conectan a recursos externos mediante 7 linked services configurados en el factory:

Linked Service	Tipo	Endpoint
AzureSqlDatabase1	Azure SQL DB	sqlservergrupo1.database.windows.net / dbRetail
AzureSqlDatabaseLinkedService	Azure SQL DB (SSL)	server--igoxkyj.database.windows.net / dbRetail
AzureBlobStorage	Azure Blob	storageblobsigoxkyj
AzureDataLakeStorage	ADLS Gen2	datalakeigoxkyj.dfs.core.windows.net (HNS habilitado)

Linked Service	Tipo	Endpoint
AzureStorageLinkedService	Azure Blob	storageblobsigoxkyj
LinkendServiceTranformaciones	Databricks	adb-4624400229247673.13.azuredatabricks.net

7.3 Distribución de Actividades

Las 15 actividades configuradas en los pipelines ADF se distribuyen entre los siguientes tipos:

Tipo de Actividad	Cantidad	Uso
Copy Activity	4	Movimiento de datos entre stores
ForEach	3	Iteración sobre listas de tablas
GetMetadata	2	Descubrimiento de archivos
SetVariable	2	Configuración dinámica
AppendVariable	1	Construcción de listas
Lookup	1	Validación de schema
Script	1	Ejecución de DDL
IfCondition	1	Branching condicional
DatabricksNotebook	1	Invocación de notebook Spark

8. Capa Analítica y de Consumo

La capa de consumo materializa el valor de negocio extraído de los datos. Está implementada en Power BI sobre la capa Gold del data lake, con un modelo dimensional optimizado para análisis interactivo.

8.1 Modelo Dimensional Star Schema

El modelo de datos sigue un esquema en estrella clásico con cuatro dimensiones principales conectadas a una tabla de hechos central:

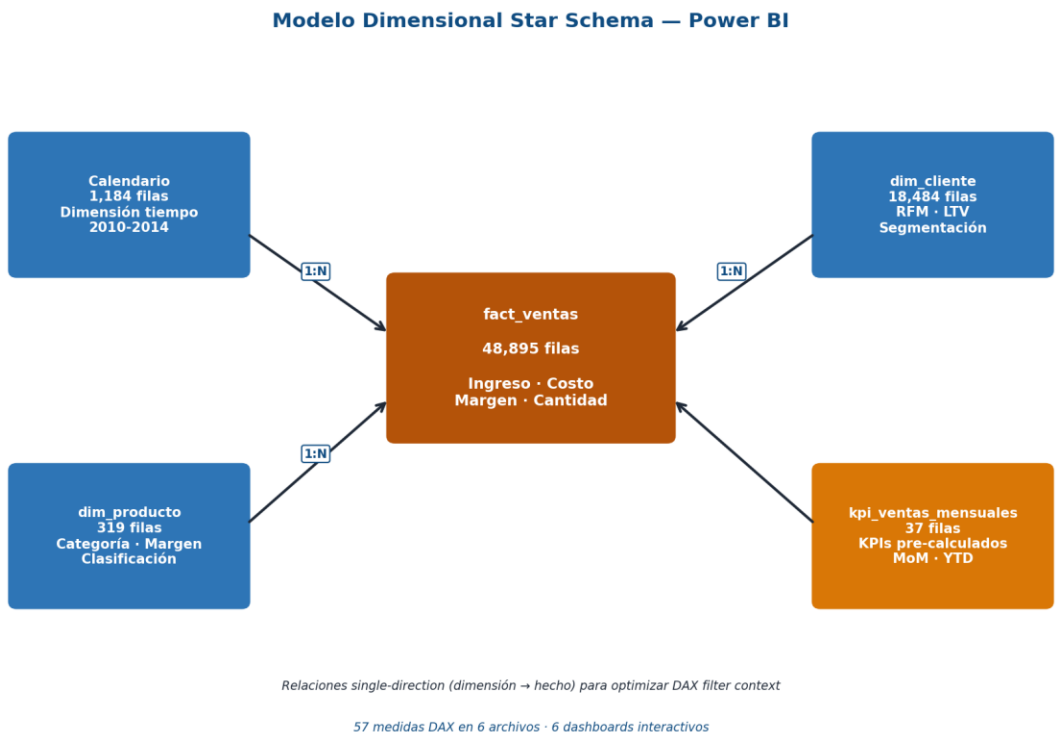

Figura 6. Modelo dimensional star schema en Power BI — tabla de hechos central y dimensiones perimetrales.

Tabla	Tipo	Filas	Origen	Rol
fact_ventas	Fact	48,895	Gold Delta Lake	Transacciones de ventas
dim_cliente	Dimensión	18,484	Gold Delta Lake	Atributos de cliente + RFM
dim_producto	Dimensión	319	Gold Delta Lake	Jerarquía de producto
kpi_ventas_mensuales	Aggregate Fact	37	Gold Delta Lake	KPIs mensuales pre-calculados

Tabla	Tipo	Filas	Origen	Rol
Calendario	Calculated	1,184	DAX ADDCOLUMNS	Dimensión tiempo (2010-2014)

Las relaciones entre tablas son siempre single direction (de dimensión a hecho), lo que optimiza el filter context evaluation de DAX y mejora el rendimiento de las medidas.

8.2 Medidas DAX

Las 57 medidas DAX se organizan en 6 archivos temáticos para facilitar mantenimiento y descubrimiento:

Archivo DAX	Medidas	Contenido Principal
01_Calendario.dax	14 columnas	Año, Mes, Trimestre, Semana, Día, Weekend, Sort
02_Medidas_Ventas_Base.dax	16-17	Ingreso, Costo, Margen, Ganancia, Impuestos, Flete
03_Time_Intelligence.dax	12-14	MoM, YoY, YTD, QTD, media móvil
04_Medidas_Producto.dax	6-8	Participación, ranking, clasificación rentabilidad
05_Medidas_Cliente.dax	8-10	RFM, LTV, frecuencia, retención
06_Medidas_KPI_Promocion.dax	7-12	Promoción, envío, conversión, formato K/M

8.3 Dashboards

El consumo final se distribuye en 6 páginas de dashboard interactivo, diseñadas para soportar el flujo natural de análisis del negocio: desde la vista ejecutiva hasta el detalle operativo.

Página	Foco	Visualizaciones
Resumen Ejecutivo	KPIs principales y tendencias	Cards + Line charts
Análisis de Ventas	Revenue por período y categoría	Bar + Treemap
Rentabilidad	Margen por producto y tier	Waterfall + Pie
Análisis de Producto	Top N ranking y participación	Bar + Donut
Análisis de Cliente	Segmentación RFM y LTV	Scatter + Pie
Promociones	Impacto de descuentos y envío	Combo + Cards

8.4 Visualizaciones Embebidas

Adicionalmente, el AnalyticsWorkflow del pipeline genera 10 charts en formato PNG mediante JFreeChart durante la ejecución del ETL. Estos artefactos se publican en docs/analytics/images/ y permiten incluir gráficos actualizados en reportes estáticos sin necesidad de Power BI.

9. Estrategia de Seguridad

La seguridad se implementa siguiendo el modelo de defensa en profundidad con seis capas concéntricas. Cada capa aporta controles independientes, de modo que el compromiso de una no expone el sistema completo.

9.1 Modelo de Capas

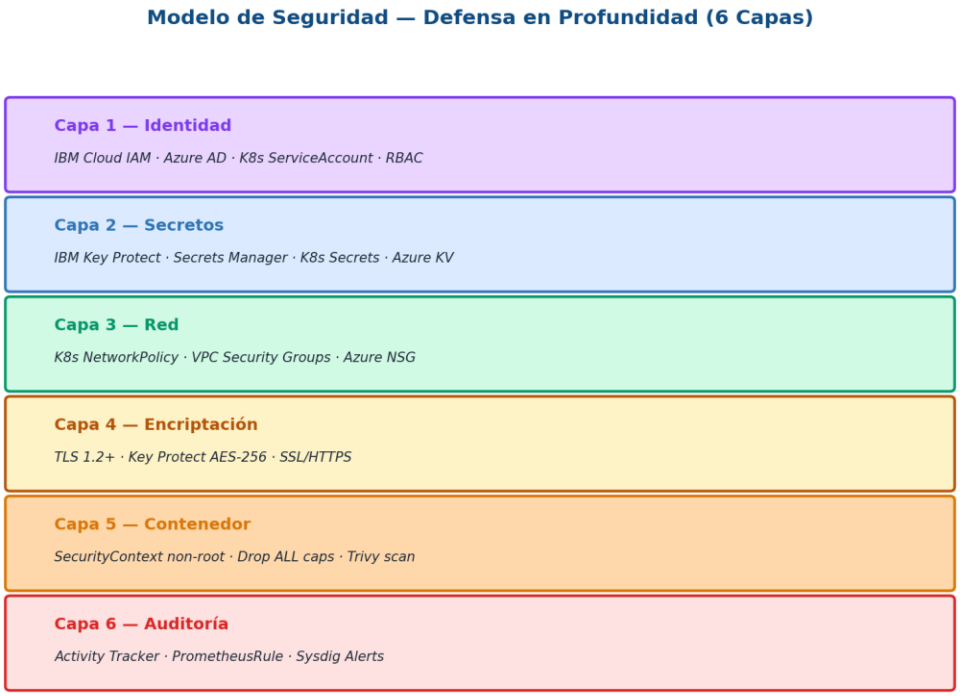

Figura 7. Modelo de defensa en profundidad — seis capas concéntricas de controles de seguridad.

Capa	Foco	Tecnologías
1. Identidad y Acceso	Autenticación y autorización	IBM Cloud IAM, Azure AD, K8s ServiceAccount
2. Secretos	Custodia de credenciales	IBM Key Protect, Secrets Manager, K8s Secrets, Azure Key Vault
3. Red	Aislamiento y tráfico controlado	K8s NetworkPolicy, IBM VPC SG, Azure NSG
4. Encriptación	Protección en tránsito y en reposo	TLS 1.2+, Key Protect AES-256
5. Contenedor	Hardening de runtime	SecurityContext non-root, drop ALL caps, Trivy scan

Capa	Foco	Tecnologías
6. Auditoría	Trazabilidad y alertas	Activity Tracker, PrometheusRule, Sysdig Alerts

9.2 Security Context de Contenedores

Todos los pods de Spark se ejecutan con un security context restrictivo que aplica el principio de mínimo privilegio:

Campo	Valor	Propósito
runAsNonRoot	true	Prohibir ejecución como root
runAsUser	185	UID del usuario spark
runAsGroup	185	GID del grupo spark
allowPrivilegeEscalation	false	Bloquear escalamiento de privilegios
readOnlyRootFilesystem	false	Spark requiere escritura en work dir
capabilities.drop	ALL	Eliminar todas las capacidades Linux

9.3 NetworkPolicy Zero-Trust

El namespace spark-medallion implementa políticas de red zero-trust que deniegan todo el tráfico por defecto y solo permiten conexiones explícitamente autorizadas:

- El driver de Spark puede crear executors mediante egress al API server de Kubernetes.
- Los executors solo se comunican con el driver: ingress y egress restringidos al rango del driver.
- El driver tiene acceso autorizado a COS por HTTPS y a Db2 por TCP en el puerto 30376.
- Tráfico entre namespaces denegado salvo excepciones explícitas para Prometheus.

9.4 Encriptación en Tránsito

Toda comunicación entre componentes utiliza canales encriptados con TLS 1.2 o superior:

Conexión	Protocolo	Configuración
SQL Server	TLS	Encrypt=True en connection string
Db2 Cloud	SSL	Puerto 30376 con sslConnection=true
IBM COS	HTTPS (S3A)	ssl.enabled=true en hadoop config
ADLS Gen2	HTTPS	Endpoint .dfs por defecto

Conexión	Protocolo	Configuración
Kubernetes API	TLS 443	ServiceAccount token

9.5 Seguridad en CI/CD

El pipeline Tekton incluye una etapa dedicada de security scan que ejecuta cuatro herramientas en paralelo. Si cualquiera detecta vulnerabilidades críticas, el pipeline se bloquea automáticamente:

Herramienta	Foco
pip-audit	Vulnerabilidades en dependencias Python
safety check	CVEs conocidos en paquetes
detect-secrets	Credenciales filtradas en código fuente
Trivy	Vulnerabilidades en imágenes de contenedor

9.6 IAM y Mínimo Privilegio

En IBM Cloud se definen dos Access Groups con permisos diferenciados según el rol del usuario:

Grupo	COS	Db2	Analytics Engine
data-engineers	Writer + Object Writer	Manager	Manager
data-analysts	Reader + Object Reader	Viewer	(sin acceso)

10. Continuous Integration y Continuous Delivery

La automatización del ciclo de vida del software se implementa con Tekton Pipelines, un framework CI/CD nativo de Kubernetes. El pipeline principal medallion-data-pipeline consta de 9 tareas que cubren desde la clonación del código hasta el deploy en los entornos de ejecución.

10.1 Etapas del Pipeline

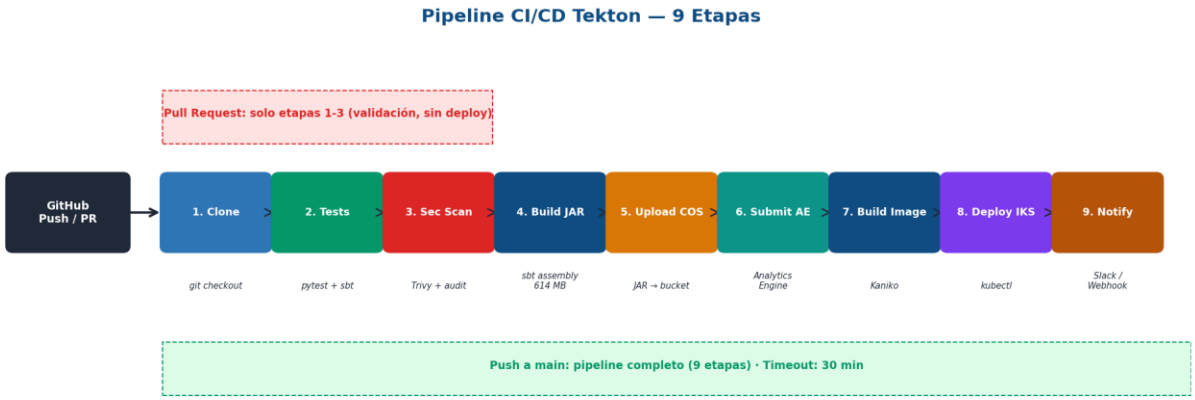

Figura 8. Pipeline Tekton CI/CD — 9 etapas con triggers diferenciados para Pull Request y Push a main.

#	Tarea	Imagen Docker	Función
1	clone-repo	alpine/git	Clona el repositorio desde GitHub
2	run-tests	python:3.12-slim + sbtscala/scala-sbt	Ejecuta pytest (Python) y sbt test (Scala)
3	security-scan	aquasec/trivy	Escanea vulnerabilidades HIGH/CRITICAL en código y Dockerfile
4	build-jar	sbtscala/scala-sbt	Compila el fat JAR (~614 MB) con sbt assembly
5	upload-to-cos	ibmcloud CLI	Sube el JAR a datalake-raw/spark-jars/
6	submit-to-ae	ibmcloud CLI	Envía el job a Analytics Engine, monitorea cada 10 s
7	build-image	gcr.io/kaniko	Construye imagen Docker y la sube a IBM Container Registry
8	deploy-to-iks	bitnami/kubectl	Aplica los manifests de Kubernetes
9	notify	curl base	Notifica a Slack o Teams el estado final del pipeline

10.2 Recursos Asignados

Cada tarea declara sus requirements y limits específicos para optimizar el uso del cluster Kubernetes:

Task	CPU Request	CPU Limit	Memory Request	Memory Limit
Tests	250m	500m	512Mi	1Gi
Security Scan	250m	500m	512Mi	1Gi
Build JAR	500m	1	1Gi	2Gi
Build Image	500m	1	1Gi	2Gi
Deploy	100m	250m	256Mi	512Mi

10.3 Triggers

El pipeline se dispara automáticamente mediante EventListeners de Tekton conectados a webhooks de GitHub. Existen dos modos de ejecución según el evento:

Evento	Pipeline Ejecutado	Timeout	Deploy
Push a main	Completo (9 tareas)	30 min	AE + IKS según parámetro
Pull Request	Parcial (etapas 1-3)	15 min	Ninguno (solo validación)

10.4 Workspaces y Parámetros

El pipeline expone parámetros configurables que permiten reutilizar la misma definición para distintos ambientes y targets:

Parámetro	Tipo	Descripción
git-url	string	URL del repositorio
git-branch	string	Branch a construir
cos-endpoint	string	Endpoint COS para upload del JAR
ae-instance-id	string	ID del Analytics Engine destino
deploy-target	string	Target de deploy: ae | iks | both
notify-webhook	string	Webhook URL para notificaciones

11. Observabilidad y Monitoreo

El stack de observabilidad combina métricas (Prometheus), logs (IBM Log Analysis), trazas de aplicación (logs estructurados Spark) y alertas accionables (Sysdig + Slack). El objetivo es detectar problemas antes de que impacten al negocio y diagnosticarlos rápidamente cuando ocurran.

11.1 Componentes de Observabilidad

Componente	Fuente	Función
ServiceMonitor	Spark driver	Scraping de métricas cada 30 s
PodMonitor	Spark executors	Métricas de pods efímeros
PrometheusRule	Reglas de alerting	7 alertas críticas configuradas
Sysdig Dashboard	IBM Cloud Monitoring	Visualización de pipeline status
Activity Tracker	IBM Cloud	Auditoría de eventos de acceso
Log Analysis	IBM Cloud	Centralización y búsqueda de logs
MetricsWorkflow	Pipeline interno	Timing, throughput y bottleneck por stage

11.2 Alertas Configuradas

Alerta	Trigger	Severidad	Acción
COS Access Denied	5 eventos en 5 min (Activity Tracker)	HIGH	Slack + Email
Spark App Failed	metric state = failed	CRITICAL	Slack + Email
Gold Bucket > 10GB	ibm_cos.bucket.bytes_used threshold	WARNING	Email
Job Stuck	Duración > 2× promedio	HIGH	Slack
Pod OOM Risk	Memory > 90%	WARNING	Slack
Crash-Loop	RestartCount > 3	CRITICAL	Slack + PagerDuty
Quota Exceeded	ResourceQuota saturado	WARNING	Email

12. Consideraciones Transversales

12.1 Idempotencia y Checkpoints

El motor DAG implementa checkpoints persistentes mediante un PipelineStateStore (LocalStateStore por defecto, escribiendo en disco JSON o COS). Cada tarea completada se marca como tal con metadatos de duración y descripción. En ejecuciones subsiguientes, las tareas ya completadas se saltan automáticamente, lo que permite reanudar pipelines fallidos sin reprocesar trabajo costoso.

La variable de entorno PIPELINE_CLEAR_CHECKPOINTS=true permite limpiar manualmente el estado para forzar una re-ejecución completa.

12.2 Feature Flags

El comportamiento del pipeline se controla mediante variables de entorno que actúan como feature flags, permitiendo activar o desactivar workflows sin redeploy:

Variable	Valores	Efecto
PIPELINE_ENV	dev | staging | prod	Determina criticidad de tareas como QUALITY
PIPELINE_CHARTS_ENABLED	true | false	Activa generación de charts JFreeChart
PIPELINE_HIVE_ENABLED	true | false	Activa registro en Hive Catalog
PIPELINE_QUALITY_MIN_SCORE	0-100	Umbral mínimo de calidad para no fallar
PIPELINE_NOTIFY_SLACK	true | false	Habilita notificaciones a Slack
PIPELINE_DAG_TIMEOUT_MIN	número (default 15)	Timeout global del DAG en minutos
PIPELINE_CLEAR_CHECKPOINTS	true | false	Limpia checkpoints anteriores

12.3 ViewScope (Loan Pattern)

Para gestionar correctamente las temporary views de Spark (createOrReplaceTempView), se implementa ViewScope siguiendo el loan pattern de Scala. Esto garantiza que las views se eliminen automáticamente al salir del scope, evitando memory leaks en pipelines de larga duración.

12.4 Estrategia de Partitioning

Las tablas Bronze, Silver y Gold se particionan estratégicamente según los patrones de consulta esperados. Las tablas de hechos se particionan típicamente por año y mes, mientras que las

dimensiones se mantienen sin particionar dada su menor cardinalidad. Delta Lake aplica adicionalmente Z-Order sobre columnas de filtrado frecuente.

12.5 Disaster Recovery

La estrategia de recuperación ante desastres se sustenta en tres pilares: (i) snapshots automáticos de Delta Lake con retención configurable que habilitan time travel y rollback de transacciones; (ii) replicación geo-redundante de los buckets COS en IBM Cloud; (iii) backup periódico del archivo terraform.tfstate a un bucket dedicado para reconstruir la infraestructura desde cero ante una pérdida total.

13. Estructura del Repositorio

El repositorio adopta una estructura monorepo que alberga código de aplicación, infraestructura como código, scripts operacionales y documentación. Esta organización facilita la trazabilidad entre cambios en distintas capas del stack.

13.1 Directorios Principales

Directorio	Contenido
transformation/spark-jobs/pipelines/batch-etl-scala/	Código fuente Scala del pipeline ETL principal
transformation/notebooks/	Notebooks PySpark para Databricks (Bronze, Silver, Gold)
infrastructure/spark-k8s/	Manifests Kubernetes, Dockerfile y Makefile para Spark on K8s
infrastructure/hadoop/	Docker Compose y configuración del lakehouse on-premise
infrastructure/ibm-cloud/	Terraform, Tekton pipelines, scripts de deploy y monitoring
infrastructure/ibm-cloud/terraform/	Definiciones IaC: main.tf, variables, locals, outputs, backend
infrastructure/ibm-cloud/tekton/	Definiciones de Pipelines, Tasks y Triggers de Tekton
infrastructure/scripts.ps1 / script.ps1	PowerShell de provisionamiento Azure
infrastructure/powerbi-export/	Archivos DAX y assets para Power BI
docs/	Documentación técnica modular con diagramas Mermaid

13.2 Documentación Modular

La documentación está organizada en módulos temáticos bajo el directorio docs/, totalizando aproximadamente 62 diagramas Mermaid distribuidos en estos componentes:

Categoría	Componentes Documentados	Diagramas
Arquitectura	Topología, Medallion flow, modos de ejecución	4
Kubernetes	CronJob, RBAC, NetworkPolicy, monitoring, Docker	8
Hadoop	HDFS, Hive, Docker Compose, estructura de datos	5
IBM Cloud	Terraform, scripts, Analytics Engine, monitoreo	5
Azure	ADF, SQL Server, provisionamiento, linked services	4
Orquestación	7 pipelines ADF, actividades, flujo completo	8
ETL Pipeline	DAG engine, circuit breaker, capas, workflows	9

Categoría	Componentes Documentados	Diagramas
Power BI	Star schema, 57 medidas, relaciones, dashboards	6
Seguridad	Secrets, RBAC, NetworkPolicy, encryption	7
CI/CD	9 stages Tekton, triggers, flujo completo	6

14. Conclusiones del Diseño

14.1 Fortalezas Arquitectónicas

El diseño analizado presenta varias decisiones arquitectónicas notables que merecen destacarse:

- **Separación clara de responsabilidades:** los seis paquetes Scala del motor ETL siguen el principio de responsabilidad única, lo que facilita la evolución independiente de cada concepto.
- **Portabilidad multi-cloud:** el mismo binario ejecuta en local, HDFS, IBM Analytics Engine, IKS o Databricks, lo que evita el vendor lock-in y facilita migraciones.
- **Resiliencia incorporada:** circuit breaker, retry con backoff exponencial, errores tipados y checkpoints persistentes son patrones de ingeniería que reducen la fragilidad operacional.
- **Catálogo único:** TableRegistry como fuente única de verdad elimina inconsistencias y simplifica el mantenimiento.
- **Seguridad multicapa:** el modelo de defensa en profundidad con seis capas independientes aplica buenas prácticas de la industria.
- **Observabilidad nativa:** métricas Prometheus, alertas Sysdig y MetricsWorkflow propio brindan visibilidad operacional completa.

14.2 Áreas Potenciales de Mejora

Se identifican algunas áreas que podrían fortalecerse en futuras iteraciones de la plataforma:

- **Streaming:** la plataforma actual es exclusivamente batch. Habilitar Structured Streaming sobre Delta Lake permitiría latencias near-real-time para casos de uso operacionales.
- **Lineage automatizado:** aunque existe LineageWorkflow propio, integrar herramientas estándar como OpenLineage o DataHub facilitaría la gobernanza enterprise.
- **Testing de integración:** complementar los tests unitarios con tests de integración end-to-end usando Testcontainers mejoraría la confianza en cambios de infraestructura.
- **Catálogo unificado:** evolucionar de Hive Metastore hacia Unity Catalog o Apache Polaris habilitaría gobernanza fine-grained de columnas y filas.
- **Service mesh:** introducir Istio o Linkerd en IKS reforzaría las políticas de red zero-trust con mTLS automático.

14.3 Madurez de la Solución

Considerando el conjunto de capacidades implementadas — patrón Medallion riguroso, motor DAG con resiliencia avanzada, despliegue multi-cloud con IaC, CI/CD automatizado con security scanning, RBAC y

NetworkPolicy zero-trust, observabilidad de extremo a extremo y modelo dimensional pulido para BI — la plataforma se sitúa en un nivel de madurez production-grade. Las decisiones arquitectónicas evidencian comprensión profunda de patrones modernos de ingeniería de datos y constituyen una base sólida para escalar a nuevos dominios y volúmenes.

La documentación interna del repositorio (aproximadamente 62 diagramas Mermaid distribuidos en módulos temáticos) refleja además un compromiso con la mantenibilidad a largo plazo y facilita el onboarding de nuevos miembros del equipo, un activo frecuentemente subestimado pero crítico para la sostenibilidad de plataformas técnicas complejas.

Fin del Documento